

SYNTH-06: Synthetic Analytics & Alerting

Series: SYNTH — Synthetic Monitoring | **Notebook:** 6 of 6 | **Created:** December 2025 | **Last Updated:** 07/30/2026

Dashboards, SLOs, and Alerting Strategies

This notebook covers advanced analytics for synthetic monitoring, including building dashboards, configuring SLOs, and implementing effective alerting strategies using the latest Dynatrace platform.

Table of Contents

1. [Availability Analysis](#)
 2. [Performance Analysis](#)
 3. [Location Comparison](#)
 4. [SLO Configuration](#)
 5. [Alerting Strategies](#)
 6. [Dashboard Building](#)
 7. [Series Complete!](#)
-

Prerequisites

-  Access to a Dynatrace environment with Synthetic Monitoring
-  Completed SYNTH-01 through SYNTH-05
-  Active synthetic monitors generating data

Query convention: These analyses span all monitor types, so they read monitor-level execution events with `filter endsWith(event.type, "_monitor_execution")` and group by `monitor.name`. Success is `result.state == "SUCCESS"`; duration is `result.statistics.duration / 1ms`.

1. Analytics Overview

Key Metrics for Synthetic Monitoring

Metric Category	Metrics	Use Case
Availability	Success rate, failure count	SLA compliance
Performance	Response time, TTFB, DNS	User experience
Reliability	Consecutive failures, MTTR	Stability
Geographic	Per-location metrics	Regional issues

Data Sources

Source	Description	Best For
<code>dt.synthetic.events</code>	Execution results (events)	Per-execution analysis, failures, percentiles
<code>dt.synthetic.detailed_events</code>	Per-step request detail	Deep-dive on individual steps
<code>timeseries dt.synthetic.*</code>	Pre-aggregated metrics	Long-term trends, dashboards
<code>dt.entity.synthetic_test</code> / <code>dt.entity.http_check</code> / <code>dt.entity.multiprotocol_monitor</code>	Monitor definitions (classic — still functional)	Configuration, name resolution

Source	Description	Best For
<code>dt.entity.synthetic_location</code>	Location info (classic — still functional)	Geographic analysis
<code>smartscapeNodes</code> "BROWSER_MONITOR" / "HTTP_MONITOR" / "NETWORK_AVAILABILITY_MONITOR"	Monitor definitions (Smartscape — preferred)	Configuration; <code>name</code> is already the display name, so no <code>entityName()</code> step
<code>smartscapeNodes</code> "BROWSER_MONITOR_STEP" / "HTTP_MONITOR_STEP"	Step definitions as separate nodes	Per-step configuration; <code>traverse</code> the <code>belongs_to</code> edge to reach the parent monitor
<code>smartscapeNodes</code> "SYNTHETIC_LOCATION"	Location info (Smartscape — preferred)	Geographic analysis with <code>location_type</code> , <code>stage</code> , <code>cloud.provider</code> , <code>geo.*</code> — none of which exist on the classic entity

Note on the entity rows. SaaS 1.344 (released 07/27/2026, **staged tenant rollout from 07/29/2026**) makes `dt.smartscape.*` the primary entity surface for the Synthetic app. The classic tables above still work, so they remain a genuine fallback while the rollout completes. Two mapping traps: the network-monitor node is `NETWORK_AVAILABILITY_MONITOR` (**not** `MULTIPROTOCOL_MONITOR`), and classic `synthetic_test` splits into `BROWSER_MONITOR` + `BROWSER_MONITOR_STEP`. See SYNTH-01 § 4 for the full mapping table and FAQ-16 for migration mechanics.

2. Availability Analysis

Calculating Availability

$\text{Availability} = (\text{Successful Executions} / \text{Total Executions}) \times 100\%$

Example SLA Targets:

- 99.9% = max 8.76 hours downtime/year
- 99.5% = max 43.8 hours downtime/year
- 99.0% = max 87.6 hours downtime/year

```
// Overall synthetic availability (last 7 days)
fetch dt.synthetic.events, from: now() - 7d
| filter endsWith(event.type, "_monitor_execution")
| summarize {
    total_executions = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
}
| fieldsAdd availability_pct = round((successful * 100.0) / total_executions,
decimals: 3)
| fieldsAdd downtime_minutes = round((failed * 5.0), decimals: 0) // Assuming
5-min frequency
```

```
// Availability by monitor (last 7 days)
fetch dt.synthetic.events, from: now() - 7d
| filter endsWith(event.type, "_monitor_execution")
| summarize {
    total = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
}, by: {monitor.name}
| fieldsAdd availability_pct = round((successful * 100.0) / total, decimals:
3)
| sort availability_pct asc
| limit 30
```

```
// Availability trend over time
fetch dt.synthetic.events, from: now() - 7d
| filter endsWith(event.type, "_monitor_execution")
| fieldsAdd hour_bucket = bin(timestamp, 1h)
| summarize {
    success_count = countIf(result.state == "SUCCESS"),
    total_count = count()
}, by: {hour_bucket}
| fieldsAdd availability_pct = round((success_count * 100.0) / total_count,
decimals: 2)
| sort hour_bucket desc
```

```
// Daily availability report
fetch dt.synthetic.events, from: now() - 30d
| filter endsWith(event.type, "_monitor_execution")
| fieldsAdd day = formatTimestamp(timestamp, format: "yyyy-MM-dd")
| summarize {
    total = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
}, by: {day}
| fieldsAdd availability_pct = round((successful * 100.0) / total, decimals:
3)
| sort day desc
| limit 30
```

3. Performance Analysis

Performance Metrics

Metric	Description	Good	Warning	Critical
Response Time	Total execution time	< 2s	2-5s	> 5s
DNS	DNS resolution	< 50ms	50-200ms	> 200ms
Connect	TCP connection	< 100ms	100-300ms	> 300ms
TTFB	Time to first byte	< 500ms	500ms-1s	> 1s

```
// Response time percentiles by monitor
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter result.state == "SUCCESS"
| summarize {
    p50 = percentile(result.statistics.duration, 50),
    p75 = percentile(result.statistics.duration, 75),
    p90 = percentile(result.statistics.duration, 90),
    p95 = percentile(result.statistics.duration, 95),
    p99 = percentile(result.statistics.duration, 99),
    executions = count()
}, by: {monitor.name}
| fieldsAdd p50_ms = round(p50/1ms, decimals:1), p75_ms = round(p75/1ms,
decimals:1),
    p90_ms = round(p90/1ms, decimals:1), p95_ms = round(p95/1ms,
decimals:1),
    p99_ms = round(p99/1ms, decimals:1)
| fieldsRemove p50, p75, p90, p95, p99
| sort p95_ms desc
| limit 20
```

```
// Performance trend comparison (this week vs last week)
fetch dt.synthetic.events, from: now() - 14d
| filter endsWith(event.type, "_monitor_execution")
| filter result.state == "SUCCESS"
| fieldsAdd week = if(timestamp > now() - 7d, "This Week", else: "Last
Week")
| summarize {
    avg_response_ms = round(avg(result.statistics.duration / 1ms), decimals:
1),
    p95 = percentile(result.statistics.duration / 1ms, 95),
    executions = count()
}, by: {monitor.name, week}
| fieldsAdd p95_response_ms = round(p95, decimals: 1)
| fieldsRemove p95
| sort monitor.name asc, week asc
```

```
// Timing breakdown analysis (HTTP step records)
// TTFB already includes DNS + TCP + TLS, so these phases overlap – do not sum
them.
fetch dt.synthetic.events, from: now() - 24h
| filter event.type == "http_step_execution"
| filter result.status.code == 0
| summarize {
    avg_dns_ms      = round(avg(result.statistics.host_name_resolution_time) /
1ms, decimals: 2),
    avg_connect_ms  = round(avg(result.statistics.tcp_connect_time) / 1ms,
decimals: 2),
    avg_tls_ms      = round(avg(result.statistics.tls_handshake_time) / 1ms,
decimals: 2),
    avg_ttfb_ms     = round(avg(result.statistics.time_to_first_byte) / 1ms,
decimals: 2),
    avg_total_ms    = round(avg(result.statistics.duration) / 1ms, decimals: 2)
}, by: {monitor.name}
| sort avg_total_ms desc
| limit 20
```

```
// Performance anomalies (max response time > 2x average per monitor)
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter result.state == "SUCCESS"
| fieldsAdd response_ms = result.statistics.duration / 1ms
| summarize {
    avg_response = avg(response_ms),
    max_response = max(response_ms),
    min_response = min(response_ms),
    executions = count()
}, by: {monitor.name}
| fieldsAdd deviation_factor = round(max_response / avg_response, decimals: 1)
| filter deviation_factor > 2
| sort deviation_factor desc
| limit 50
```

4. Location Comparison

Geographic Analysis

Compare synthetic results across locations to:

- Identify regional performance issues

- Detect CDN or DNS problems
- Validate global availability
- Optimize user experience by region

```
// Availability by location
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| summarize {
    total = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
}, by: {dt.entity.synthetic_location}
| fieldsAdd availability_pct = round((successful * 100.0) / total, decimals:
2)
| fieldsAdd location = entityName(dt.entity.synthetic_location)
| sort availability_pct asc
| limit 30
```

```
// Performance by location
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter result.state == "SUCCESS"
| summarize {
    avg_response_ms = round(avg(result.statistics.duration / 1ms), decimals:
1),
    p50 = percentile(result.statistics.duration, 50),
    p95 = percentile(result.statistics.duration, 95),
    executions = count()
}, by: {dt.entity.synthetic_location}
| fieldsAdd p50_ms = round(p50 / 1ms, decimals: 1), p95_ms = round(p95 / 1ms,
decimals: 1)
| fieldsRemove p50, p95
| fieldsAdd location = entityName(dt.entity.synthetic_location)
| sort avg_response_ms desc
| limit 30
```



```
// Location performance heatmap (monitor x location)
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| summarize {
    executions = count(),
    availability_pct = round(countIf(result.state == "SUCCESS") * 100.0 /
count(), decimals: 2),
    avg_response_ms = round(avg(result.statistics.duration / 1ms), decimals:
1)
}, by: {monitor.name, dt.entity.synthetic_location}
| fieldsAdd location = entityName(dt.entity.synthetic_location)
| sort monitor.name asc, avg_response_ms desc
```

```
// Location-specific failures
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter result.state == "FAIL"
| summarize {
    failure_count = count(),
    unique_monitors = countDistinct(monitor.name),
    error_types = collectDistinct(result.status.message)
}, by: {dt.entity.synthetic_location}
| fieldsAdd location = entityName(dt.entity.synthetic_location)
| sort failure_count desc
| limit 20
```

5. SLO Configuration

Service Level Objectives for Synthetic

SLO Type	Metric	Example Target
Availability	Success rate	99.9%
Performance	Response time P95	< 3000ms
Error Budget	Allowed failures	0.1% of executions

Creating Synthetic SLOs

1. **Navigate to:** Settings → Service-level objectives

2. **Create SLO:** Name, description, timeframe
3. **Define metric:** Use synthetic availability or response time
4. **Set target:** e.g., 99.9% availability
5. **Configure alerting:** Error budget alerts

```
// SLO calculation - Availability (30-day window)
fetch dt.synthetic.events, from: now() - 30d
| filter endsWith(event.type, "_monitor_execution")
| summarize {
    total_executions = count(),
    successful = countIf(result.state == "SUCCESS"),
    failed = countIf(result.state == "FAIL")
}, by: {monitor.name}
| fieldsAdd sli_availability = round((successful * 100.0) / total_executions,
decimals: 4)
| fieldsAdd slo_target = 99.9
| fieldsAdd error_budget_total = round(total_executions * 0.001, decimals: 0)
// 0.1% budget
| fieldsAdd error_budget_remaining = round(error_budget_total - failed,
decimals: 0)
| fieldsAdd error_budget_pct = round((error_budget_remaining * 100.0) /
error_budget_total, decimals: 1)
| fieldsAdd slo_status = if(sli_availability >= slo_target, "✅ Met",
else: "❌ Breached")
| sort sli_availability asc
| limit 30
```

```
// SLO calculation - Performance (P95 < 3000ms)
fetch dt.synthetic.events, from: now() - 30d
| filter endsWith(event.type, "_monitor_execution")
| filter result.state == "SUCCESS"
| fieldsAdd response_ms = result.statistics.duration / 1ms
| summarize {
    executions = count(),
    p95 = percentile(response_ms, 95),
    within_target = countIf(response_ms < 3000)
}, by: {monitor.name}
| fieldsAdd p95_response_ms = round(p95, decimals: 1)
| fieldsRemove p95
| fieldsAdd sli_performance = round((within_target * 100.0) / executions,
decimals: 2)
| fieldsAdd slo_target = 95.0 // 95% of requests < 3s
| fieldsAdd slo_status = if(sli_performance >= slo_target, "✅ Met", else:
"❌ Breached")
| sort sli_performance asc
| limit 30
```

```
// Error budget burn rate (last 7 days)
fetch dt.synthetic.events, from: now() - 7d
| filter endsWith(event.type, "_monitor_execution")
| fieldsAdd day = formatTimestamp(timestamp, format: "yyyy-MM-dd")
| summarize {
    total = count(),
    failed = countIf(result.state == "FAIL")
}, by: {monitor.name, day}
| fieldsAdd daily_error_rate = round((failed * 100.0) / total, decimals: 3)
| fieldsAdd budget_consumed = round(failed * 100.0 / (total * 0.001),
decimals: 1) // vs 0.1% budget
| sort monitor.name asc, day desc
```

6. Alerting Strategies

Alert Types

Alert Type	Trigger	Use Case
Availability	Consecutive failures	Outage detection
Performance	Response time threshold	Degradation

Alert Type	Trigger	Use Case
SLO	Error budget consumption	Proactive warning
SSL	Certificate expiration	Security

Alert Configuration Best Practices

Setting	Recommendation	Reason
Consecutive failures	2-3	Avoid false positives
Location threshold	2+ locations	Confirm not local issue
Alert delay	5-10 minutes	Allow for transients
Auto-resolve	After 2 successes	Clear resolved issues

```
// Monitors with recent failures (potential outages)
fetch dt.synthetic.events, from: now() - 1h
| filter endsWith(event.type, "_monitor_execution")
| filter result.state == "FAIL"
| summarize {
    failure_count = count(),
    last_failure = max(timestamp),
    first_failure = min(timestamp)
}, by: {monitor.name, dt.entity.synthetic_location}
| filter failure_count >= 2
| fieldsAdd location = entityName(dt.entity.synthetic_location)
| sort failure_count desc
| limit 20
```

```
// Performance degradation alerts (P95 > threshold)
fetch dt.synthetic.events, from: now() - 1h
| filter endsWith(event.type, "_monitor_execution")
| filter result.state == "SUCCESS"
| summarize {
    avg_response_ms = round(avg(result.statistics.duration / 1ms), decimals:
1),
    p95 = percentile(result.statistics.duration / 1ms, 95),
    executions = count()
}, by: {monitor.name}
| fieldsAdd p95_response_ms = round(p95, decimals: 1)
| fieldsRemove p95
| filter p95_response_ms > 5000 // > 5 seconds threshold
| sort p95_response_ms desc
```

```
// Multi-location failure detection (global outage indicator)
fetch dt.synthetic.events, from: now() - 30m
| filter endsWith(event.type, "_monitor_execution")
| filter result.state == "FAIL"
| summarize {
    failing_locations = countDistinct(dt.entity.synthetic_location),
    failure_count = count(),
    locations = collectDistinct(dt.entity.synthetic_location)
}, by: {monitor.name}
| filter failing_locations >= 2
| fieldsAdd severity = if(failing_locations >= 3, "CRITICAL", else:
"WARNING")
| sort failing_locations desc
```

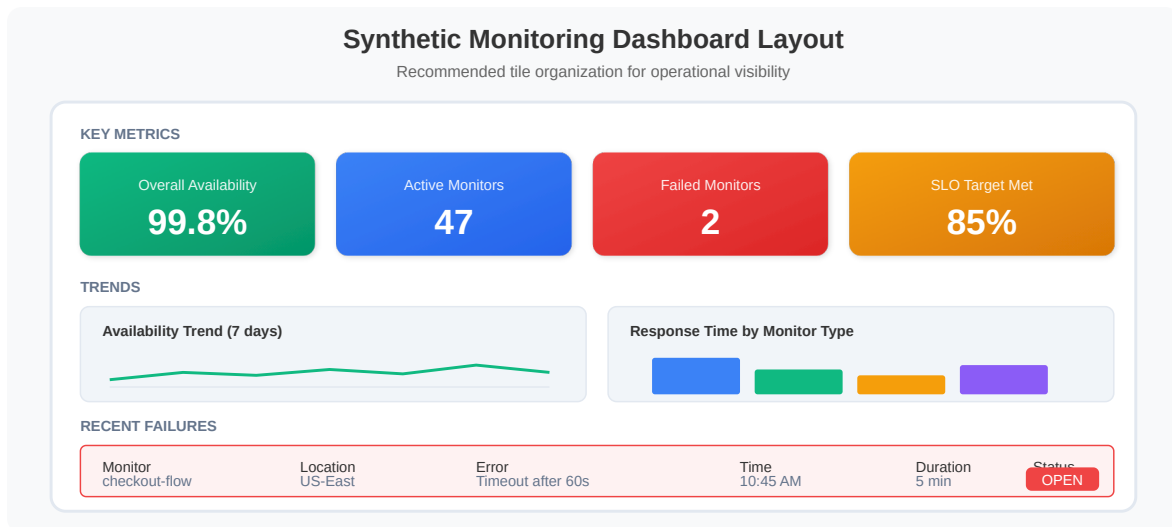
7. Dashboard Building

Recommended Dashboard Tiles

Tile Type	Content	Visualization
Overall Health	Availability %	Single value
Availability Trend	Hourly availability	Line chart
Response Time	P95 by monitor	Bar chart
Location Map	Geographic availability	World map

Tile Type	Content	Visualization
Failures Table	Recent failures	Table
SLO Status	Error budget	Gauge

Dashboard Layout



```
// Dashboard tile: Overall availability (single value)
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| summarize {
  availability_pct = round(countIf(result.state == "SUCCESS") * 100.0 /
count(), decimals: 2)
}
```

```
// Dashboard tile: Healthy vs failing monitors (last hour)
fetch dt.synthetic.events, from: now() - 1h
| filter endsWith(event.type, "_monitor_execution")
| summarize {
  failed_recent = countIf(result.state == "FAIL"),
  executions = count()
}, by: {monitor.name}
| summarize {
  total_monitors = count(),
  healthy = countIf(failed_recent == 0),
  failing = countIf(failed_recent > 0)
}
```

```
// Dashboard tile: Response time by monitor (bar chart)
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter result.state == "SUCCESS"
| summarize {
    p95 = percentile(result.statistics.duration / 1ms, 95)
}, by: {monitor.name}
| fieldsAdd p95_response_ms = round(p95, decimals: 1)
| fieldsRemove p95
| sort p95_response_ms desc
| limit 10
```

```
// Dashboard tile: Recent failures table
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| filter result.state == "FAIL"
| fields timestamp,
    monitor = monitor.name,
    location = entityName(dt.entity.synthetic_location),
    status = result.status.message,
    detail = result.status.details
| sort timestamp desc
| limit 20
```

```
// Dashboard tile: Hourly availability trend (line chart)
fetch dt.synthetic.events, from: now() - 24h
| filter endsWith(event.type, "_monitor_execution")
| fieldsAdd hour_bucket = bin(timestamp, 1h)
| summarize {
    success_count = countIf(result.state == "SUCCESS"),
    total_executions = count(),
    failures = countIf(result.state == "FAIL")
}, by: {hour_bucket}
| fieldsAdd availability_pct = round((success_count * 100.0) /
total_executions, decimals: 2)
| sort hour_bucket asc
```

Summary

In this notebook, you learned:

- ✓ **Availability analysis** - Calculating and trending availability with `result.state`
 - ✓ **Performance analysis** - Percentiles, timing breakdown, anomalies
 - ✓ **Location comparison** - Geographic performance analysis
 - ✓ **SLO configuration** - Service level objectives and error budgets
 - ✓ **Alerting strategies** - Outage detection, performance alerts
 - ✓ **Dashboard building** - Key tiles and layouts
-

Series Complete!

You have completed the Synthetic Monitoring Best Practices series:

1. ✓ **SYNTH-01**: Fundamentals
 2. ✓ **SYNTH-02**: Browser Monitors
 3. ✓ **SYNTH-03**: HTTP Monitors
 4. ✓ **SYNTH-04**: Private Locations
 5. ✓ **SYNTH-05**: Network Monitoring
 6. ✓ **SYNTH-06**: Analytics & Alerting
-

References

- [Synthetic on Grail \(DT docs\)](#)
 - [Synthetic alerting overview \(DT docs\)](#)
 - [Synthetic for Workflows \(DT docs\)](#)
 - [Service-level objectives \(DT docs\)](#)
 - [Dashboards \(DT docs\)](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.